

Allokering och frigöring av minne

För att kunna allokera och frigöra minne dynamiskt måste man i C++ ha något "ställe" dit minnet kan placeras då det allokateras. I C++ används pekare för att referera till allokerat minne. Via pekaren kan minnet sedan avleveras och användas.

Allokering av minne

Allokering av minne sker med operatoren `new`. Allmänt ser en allokering ut på följande sätt:

```
datatyp * pekarnamn = new datatyp;
```

Man bör alltså ha en pekare till den datatypen man ämnar allokera minne för, t.ex. `int`. Operatoren `new` påminner om en funktion, men det är egentligen en operator, så den behöver inte parenteser runt sin parameter. Datatypen som ges till `new` måste vara av samma typ som den pekare som skall ta emot det minne som `new` returnerar. Efter en allokering kan man avreferera den använda pekaren och använda denna som vilken variabel som helst. Några exempel på allokeringar:

```
int * IntPekare = new int;
float * FloatPekare;
FloatPekare = new float;
string * Text = new string;
```

Man behöver alltså inte direkt allokera minne till en pekare, utan man kan först deklarera en pekare och sedan allokera minne i ett senare skede. Man kan även allokera minne flera gånger med hjälp av samma pekare. Flera pekare kan även peka på samma minnesområde, precis som flera pekare kan peka på samma variabls minnesadress. Man bör dock observera att om det finns endast en pekare till ett allokerat minnesblock, och denna pekare sätts att peka på något annat, så har man tappat det allokerade minnet. Det finns därefter *ingen* möjlighet att hitta det på nytt. Man har *läckt minne*. Se mera info om minnesläckor i [avsnittet Minnesläckor](#).

Vad händer om vårt program allokerar för mycket minne och det tar slut? Detta är ganska sällsynt, men kan hända om minnesläckor förekommer eller om maskinen har litet minne. I så fall kommer `new` att "kasta" en *exception* som heter `bad_alloc`. Mera om exceptions i [Kapitel 20](#), där exempel på en minnes-exception visas.

Allokera vektorer

Förutom att man kan allokera enskilda variabler, såsom en enskild `int`, kan man även allokera en vektor av en och samma datatyp med `new`. Den allmänna formen ser då ut så här:

```
datatyp * pekarnamn = new datatyp [antal];
```

Enda skillnaden från när man allokerar en primitiv är att man specificerar antalet element som vektorn skall innehålla inom `[ ]`. Denna storlek är exakt det antal element i vektorn som kan användas. Exempel på några vektorallokeringar:

```
int * Buffer = new int [1024 * 1024];
float * Koordinat;
Koordinat = new float [3];
string * Brev = new string [100];
```

Man kan allokera mycket stora minnesblock som vektorer. Den enda begränsningen är mängden minne i datorn som används. Det första exemplet allokerar en buffer som är över en miljon element stor, och eftersom en `int` är troligtvis fyra bytes stor (eller större) har man allokert över fyra miljoner bytes (4 MB). Precis som med normala vektorer bör man inte avreferera element som är utanför det allokerade minnesområdet. Om man t.ex. allokerar 10 element är element 0 det första och element 9 det sista som kan användas, precis som med normala vektorer.

De allokerade vektorerna kan sedan användas precis som normala vektorer, utan speciell avreferering el.dyl., eftersom vektorer i sig redan är pekare. Nedan finns ett exempel som dynamiskt allokerar en vektor och fyller denna med data:

```
#include <iostream>

void in (int * Tal, unsigned int Antal) {
    // iterera och läs in tal
    for ( unsigned int Index = 0; Index < Antal; Index++ ) {
        cout << "Ge in ett tal: ";
        cin >> Tal[Index];
    }
}

void ut (int * Tal, unsigned int Antal) {
    // iterera och skriv ut våra tal
    for ( unsigned int Index = 0; Index < Antal; Index++ ) {
        cout << "Talet är: " << Tal[Index] << endl;
    }
}

int main () {

    unsigned int Antal;
    int * Vektor;

    // hur många tal önskas?
    cout << "Antalet tal i vektorn: ";
    cin >> Antal;

    // allokera en vektor för att hålla alla tal
    Vektor = new int [ Antal ];

    // läs in data och skriv sedan ut igen
    in ( Vektor, Antal );
    ut ( Vektor, Antal );
}
```

Processen att hantera dynamiska vektorer är precis likadan som för statiska vektorer, med enda undantaget sättet på vilket de skapas.

Allokering av sammansatta datatyper

Som man kunde gissa är det inget speciellt med att allokera sammansatta datatyper. De fungerar precis på samma sätt som övrig allokering. Om vi t.ex. tittar på den `struct Person` som vi skapade i [avsnittet Sammansatta datatyper i Kapitel 9](#). Den såg ut så här:

```
struct Person {
    string Namn;
    string Adress;
    int    Telefonnummer;
};
```

För att allokeras ett element av typen `Person` kan man göra följande:

```
Person * P = new Person;
P->Namn = "Joe Foo";
P->Adress = "DataCity";
P->Telefonnummer = 5551234;
```

Noter att vi här använder oss av `->` för att avreferera en pekare till en `struct` istället för avreferering med `*` och punkt. Mera information om detta i [avsnittet Sammansatta datatyper i Kapitel 9](#). Man kan även sätta alla värden på samma gång:

```
Person * P = new Person { "Joe Foo", "DataCity", 5551234 };
```

Vi skall nu skapa ett litet program som hanterar en *stack* som innehåller element av datatypen `char`. Vi vill göra stacken totalt dynamisk så att man kan sätta till och ta bort element enligt önskan. Detta exempel visar hur man kan använda pekare tillsammans med `struct`. Varje element i stacken bör innehålla en `char`, samt en pekare till elementet under. Vi definierar detta element på följande sätt:

```
struct Element {
    // data för detta element
    char Data;
    // nästa element under detta i stacken
    Element * Previous;
};
```

Vi har alltid en pekare `Previous` som pekar på elementet under. Vi får således en enkel länkad lista. Första elementet har `Previous = 0`. Då man lägger till ett element kommer det alltid överst på stacken, och dess `Previous` sätts till att peka på det tidigare översta elementet. En enkel stack behöver funktioner för att lägga till ett element på stacken (`push()`), ta bort första elementer (`pop()`) samt en funktion för att testa om stacken är tom (`isEmpty()`). De tre funktionerna kan vi implementera på följande sätt:

```
#include <iostream>

struct Element {
    // data för detta element
    char Data;
    // nästa element under detta i stacken
    Element * Previous;
};

// push: a ett element på stacken
Element * push (Element * Top) {
    char Data;

    cout << "Elementets värde: ";
    cin >> Data;

    // skapa nytt element
    Element * New = new Element;
    New->Data = Data;
```

```

New->Previous = Top;

// återvänd och returnera nya topp-elementet
return New;
}

// pop:a ett element från stacken
Element * pop (Element * Top, char & Data) {
    // är stacken tom?
    if ( Top == 0 ) {
        // stacken tom
        return 0;
    }

    // spara data som är lagret i elementet
    Data = Top->Data;

    // spara pekare till det element som blir nytt topp-element
    Element * Tmp = Top->Previous;

    // frigör minne och återvänd
    delete Top;
    return Tmp;
}

// kolla om stacken är tom
bool isEmpty (Element * Top) {

    if ( Top == 0 ) {
        // stacken tom
        return true;
    }

    // stacken inte tom
    return false;
}

```

Koden ovan är relativt enkel att förstå. Både `push()` och `pop()` modifierar stacken, så de returnerar därför den nya stacken. Anroparen måste därför "fånga upp" detta returvärde för att undvika fel. En tom stack har alltid värdet 0, eftersom det då inte finns element allokerade. `pop()` returnerar 0 om stacken blir tom. Funktionerna allokerar minne dynamiskt vid en `push()` och frigör det efter en `pop()` (se [avsnittet Frigöring av minne](#) för mera information om frigöring av minne). Vi behöver ännu en funktion `main()` som skall använda vår nya stack. Den kan se ut enligt följande:

```

int main () {
    Element * Top = 0;
    char Choice = ' ';
    char Data;

    // iterera tills användaren vill avsluta
    while ( Choice != '3' ) {
        cout << "Välj: " << endl << "1 - push" << endl << "2 - pop" << endl;
        cout << "3 - sluta " << endl << "-> ";

        // läs in ett menyval
        cin >> Choice;

        // vad vill användaren göra
        switch ( Choice ) {
            case '1' : Top = push ( Top ); break;
            case '2' :
                // har vi element i listan?

```

```

    if ( ! isEmpty ( Top ) ) {
        // ja, poppa och skriv ut det poppade värdet
        Top = pop ( Top, Data );
        cout << "Poppade: " << Data << endl;
    }
    else {
        cout << "Stacken tom!" << endl;
    }
    break;
}
}
}

```

Här har vi en `while`-loop som itereras ända tills användaren vill avsluta programmet. En liten meny visas där användaren kan välja att lägga eller ta bort ett element. Då vi tar bort ett element kan vi inte göra det ur en tom stack.

Denna stack är relativt begränsad, eftersom den endast kan hantera data av typen `char`, och koden måste ändras för att kunna användas för andra datatyper. Vi kommer senare att förbättra vårt stack-program och göra det mera flexibelt!

Frigöring av minne

Den största skillnaden mellan normala variabler (även kallade *automatiska*, eftersom de skapas och förstörs automatiskt) är att variabler som allokerats dynamiskt måste även *frigöras*. I detta fall skiljer sig C++ från t.ex. Java, där systemet automatiskt frigör data som inte används längre. Detta system kallas *garbage collecting*. I C++ är det programmerarens uppgift. Denna uppgift kan vålla en hel del huvudbry för ovana programmerare. Med frigöring av minne menas att ett minnesblock som är allokerat med `new` ges tillbaka till operativsystemet för att användas till något annat ändamål. Om man inte frigör använt minne kommer minnet förr eller senare att ta slut.

I exemplet ovan frigjordes inget minne, är programmet då dåligt? Nej, eftersom operativsystemet automatiskt frigör allt minne som finns allokerat då ett program terminerar. För små program med kort körtid fungerar denna approach. Man kan helt enkelt lämna frigörandet av minne till operativsystemet. Om det däremot är frågan om ett program som kommer att ha en lång körtid (t.ex. en *demon*) eller som använder mycket stora minnesmängder (t.ex. ett grafikhanteringsprogram) är det viktigt att frigöra minne. Mera info om minnesläckor i [avsnittet Minnesläckor](#).

Minne frigörs med operatorn `delete`. Eftersom det är frågan om en *operator* fungerar den på samma sätt som `new`. Allmänt frigörs minne på följande sätt:

```

delete variabel;
delete [] vektor;

```

Man kan alltså frigöra både enkla allokerade variabler och vektorer. Enda skillnaden är att man placerar en tom `[]` efter `delete` då man avser en vektor. Några exempel (vi antar att alla pekare är initialiserade med `new`):

```

string * Text = new string;
delete Text;
int * Buffer = new int [4096];
delete [] Buffer;

```

Notera att en vektor som innehåller endast ett element ändå är en vektor. Operatorn `delete` får endast användas på pekare som har returnerats av `new`, eller så på `0`. Om `delete` appliceras på `0` har

operationen ingen verkan. Om däremot `delete` appliceras på en pekare med ett slumpmässigt värde uppkommer troligtvis felsituationer. T.ex. följande är fel:

```
int main () {  
    int * A;  
    delete A;  
}
```

Vi kan inte vara säkra på att `A` har ett definierat värde, eller mera specifikt, att den har värdet 0. Om `A` har värdet 0 så händer ingenting, men om `A` har ett slumpmässigt värde försöker vi troligtvis frigöra minne som inte allokerats. Vi kunde korrigera vårt program från ovan så att det explicit frigör allokerat minne, vilket är bra att bli van med:

```
int main () {  
    unsigned int Antal;  
    int * Vektor;  
  
    // hur många tal önskas?  
    cout << "Antalet tal i vektorn: ";  
    cin >> Antal;  
  
    // allokerar en vektor för att hålla alla tal  
    Vektor = new int [ Antal ];  
  
    // läs in data och skriv sedan ut igen  
    in ( Vektor, Antal );  
    ut ( Vektor, Antal );  
  
    // frigör minne explicit  
    delete [] Vektor;  
}
```

Endast `main()` visades, de övriga funktionerna är oförändrade.

[Förutgående](#)

Dynamisk minneshantering

[Hem](#)

[Upp](#)

[Nästa](#)

Vanliga fel